


```
    struct point pt2;  
}; rt, *rp = &rt;
```

则下面不正确的引用是 _____ 。

- A、rt.pt1.x B、(*rp).pt1.x C、rp->pt1.x D、rt->pt1.x

8、若 $x = 2$, $y = 3$, 则 $x \& y$ 的结果是 _____ 。

- A、0 B、2 C、3 D、5

9、以下叙述中不正确的是 _____ 。

- A、在不同的函数中可以使用相同名字的变量。
B、函数中的形式参数是局部变量。
C、在一个函数内的复合语句中定义变量在本函数范围内有效。
D、在一个函数内定义的变量只在本函数范围内有效。

10、已知程序片段如下：

```
struct st  
{ int n;  
  struct st *next;  
};  
static struct st a[3] = { 5, &a[1], 7 &a[2], 9, '\0'}, *p;  
p = &a[0];
```

则值为 6 的表达式是 _____ 。

- A、p++ -> n B、p -> n++ C、(*p).n++ D、++p -> n

二、程序填空题（共 10 分，每空 1 分）

（在下面的横线空白处填上适当的内容，使程序能够正确完成相应功能）

已知在当前文件夹中保存有名为“A.DAT”和名为“B.DAT”的文本文件，文件中保存着升序排列的字符，两个文件中最后一个字符是'\0'。下面程序的功能是将两个文件中的内容保存在名为“C.DAT”的文本文件中，字符的保存顺序仍按从小到大升序排列，且相同的字符在文件中只保存一个，文件的最后一个字符仍是字符串结束标志。

```
#include "stdio.h"  
main( )  
{ char a, b, c, d;  
  FILE *fn1, *fn2, *fn3;  
  if( ( fn1 = fopen( "A.DAT", "r" ) ) == NULL )  
  { printf( "cannot open A.DAT! " ); exit(1); }  
  if( ( fn2 = fopen( "B.DAT", "r" ) ) == NULL )  
  { printf( "cannot open B.DAT! " ); exit(2); }  
  if( ( fn3 = fopen( "C.DAT", 【1】 ) ) == NULL )
```

```

{ printf( "cannot open C.DAT! " ); exit(3); }
a = fgetc( fn1 );
b = fgetc( fn2 );
__【2】__ = a + b;
while( __【3】__ )
{ c = a < b ? a : b;
  if( __【4】__ )
  { fputc( c, fn3 );
    d = c;
  }
  __【5】__ ? ( a = fgetc( fn1 ) ) : ( b = fgetc( fn2 ) );
}
while( __【6】__ )
{ if( __【7】__ )
  { fputc( a, fn3 );
    d = a;
  }
  a = fgetc( fn1 );
}
while( __【8】__ )
{ if( __【9】__ )
  { fputc( b, fn3 );
    d = b;
  }
  b = fgetc( fn2 );
}
fputc( __【10】__ , fn3 );
fclose( in1 );
fclose( in2 );
fclose( out );
}

```

- | | | | |
|---------------|-----------|-----------|-----------|
| 【1】 A、 "a" | B、 "b" | C、 "r" | D、 "w" |
| 【2】 A、 a | B、 b | C、 c | D、 d |
| 【3】 A、 a | B、 b | C、 a b | D、 a && b |
| 【4】 A、 a != d | B、 b != d | C、 c != d | D、 a == d |
| 【5】 A、 a < b | B、 a > b | C、 a == b | D、 a != b |
| 【6】 A、 a | B、 b | C、 c | D、 0 |
| 【7】 A、 a != d | B、 b != d | C、 c != d | D、 a == d |
| 【8】 A、 a | B、 b | C、 c | D、 '\0' |
| 【9】 A、 a != d | B、 b != d | C、 c != d | D、 a == d |

三、上机编写程序题（共 40 分，每题 10 分）

1. 输入 n 值，输出下例（ $n=4$ ）所示的边长为 n 的正六边形。（10 分）

```

*****
 *      *
 *      *
 *      *
 *      *
 *      *
*****

```

输入：边长 n

输出：边长为 n 的六边形

2. 输出阿姆斯特朗数。

如果一个 n 位正整数等于其各位数字的 n 次方之和，则称该数为阿姆斯特朗数。如 $407=4^3+0^3+7^3$ 就是一个阿姆斯特朗数。编程，输入正整数的位数 n ($n<10$)，输出 n 位正整数中的所有阿姆斯特朗数，若不存在阿姆斯特朗数，则输出：No output。（10 分）

输入：正整数的位数 n

输出：所有阿姆斯特朗数

3. 编写一个函数，使用递归算法求给定十进制正整数转换为其他进制数。

函数的原型：`dtos(long k, int t, char *sa, int *flag)`

其中：参数 k 是给定的正整数；参数 t 是给定的新的进制（例如三进制、七进制）；参数 sa 是保存新进制数的字符数组的首地址；参数 $flag$ 是标志指针，用于检测所提交的函数是否使用了递归调用。

注意：所提交的函数中，在变量说明语句后，其他可执行语句前写 TEST 四个大写英文字母。此四个大写英文字母必须写在正确的位置，否则此题不能得分。

4. 编写一个函数，将给定的单向环型链表拆分为两个单向环型链表，拆分的原则是给定某个节点的编号，从该节点到原链表的尾节点为一个链表，其他节点为一个链表。给定的链表和拆分后的链表都没有头节点；原链表的头指针所指向的节点编号为 1，节点编号依指针方向依次增加，编号最大的为尾节点。原链表的头指针为拆分后的一个环型链表的头指针，第二个环型链表的头指针指向给定编号的节点。

结构的定义：

```

struct node
{
    int num;
    struct node *next;
}

```

函数的原型: `struct node *splttlist(struct node *head, int m)`

其中: 参数 `head` 是链表的头指针; 参数 `m` 是给定节点的编号。函数的返回值是第二个节点的头指针。